SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai – 602105

A MODULAR PYTHON FRAMEWORK FOR INTELLIGENT PLANT RECOMMENDATION AND FARMING DECISION SUPPORT

A CAPSTONE PROJECT REPORT

A Capstone Project Report submitted in partial fulfilment of the requirements for the Course of

CSA 0808 – PYTHON PROGRAMMING

to the award of the degree of

BACHELOR OF TECHNOLOGY

IN

INFORMATION TECHNOLOGY

Submitted by

S. DEVADHARSHINI (192521256)

Under the Supervision of

Dr. PONMANIRAJ SAMBASIVAM

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai – 602105

SEPTEMBER 2026

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled “A Modular Python Framework for Intelligent Plant Recommendation and Farming Decision Support” has been carried out by the S. Devadharshini (192521256) under the supervision of Dr. PONMANIRAJ.S and is submitted in partial fulfilment of the requirements for the current semester of the B.Tech Information Technology program at Saveetha Institute of Medical and Technical Sciences, Chennai.

COURSE COORDINATOR	COURSE FACULTY
Dr. S. Loganayagi, Professor	Dr. Ponmaniraj.S, Professor
Computer Science and Engineering	Computational Intelligence
SIMATS Engineering	SIMATS Engineering

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

DECLARATION

I S. Devadharshini of the btech information technology, SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled "[Project Title]" is the result of our own bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with the principles of engineering ethics and academic integrity. All sources, references, data, and other materials used in the project have been appropriately acknowledged. We further declare that the data, results, analysis, and findings presented in this report have not been fabricated, falsified, or manipulated. Any external tools, software, datasets, computational resources, or AI-assisted tools used during the project have been appropriately disclosed. We confirm that all applicable ethical, academic, institutional, and professional requirements have been duly followed throughout the planning, execution, analysis, and documentation of the project.

Place: CHENNAI
Date: 05/09/2026

Signature of the Student with Name

S. Devadharshini

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

INDIVIDUAL CONTRIBUTION STATEMENT

Student Name / Register No.	Specific Responsibilities	Design & Development Contribution	Testing & Analysis Contribution	Report Contribution	Approx. Contribution (%)
S. DEVADHARSHINI (192521256)	Data preprocessing and recommendation framework	Designed the modular Python data-processing and recommendation modules.	Tested preprocessing, prediction and recommendation outputs.	Contributed to methodology, implementation and analysis sections	50%
B. SOUJANYA (192521389)	Machine-learning model development and evaluation	Developed the crop/plant recommendation model and evaluation workflow.	Tested model performance and analysed recommendation results.	Contributed to model, results and documentation sections	50%
Total					100%

ABSTRACT

Agricultural decision making depends on the interaction between soil characteristics, climatic conditions, crop requirements, water availability and cultivation practices. Farmers and agricultural learners often need to compare several parameters before selecting a suitable crop or plant. Manual comparison of such information can be time consuming and may lead to inconsistent decisions when the available agricultural data is incomplete or stored in different formats.

This project proposes a Modular Python Framework for Intelligent Plant Recommendation and Farming Decision Support. The framework is organised as independent modules for user input, agricultural knowledge storage, data processing, plant recommendation and farming decision support. Module 2, the Agricultural Knowledge Repository and Data Processing Layer, provides the data foundation for the complete system. It stores structured plant and agricultural information and converts raw records into validated, standardised and analysis-ready data.

The methodology includes data collection, schema design, validation, missing-value handling, duplicate removal, unit standardisation, feature engineering and preparation of processed records for the recommendation engine. Python libraries such as pandas and NumPy can be used to implement the processing layer, while CSV, JSON or SQLite can be used for structured storage. A rule-based suitability score can combine soil pH, temperature, rainfall, humidity, sunlight, water requirements and nutrient conditions to support transparent recommendations.

The proposed framework is intended as a decision-support system rather than a replacement for agricultural experts. Its modular architecture allows the repository and processing layer to be improved independently and provides a clear path for future integration with machine-learning models, real-time weather data and farm-specific information.

Keywords: Python, Plant Recommendation, Agriculture, Decision Support, Knowledge Repository, Data Processing, pandas, NumPy, Soil, Crop Suitability

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

LIST OF ABBREVIATIONS / SYMBOLS

Symbol	Description	Unit
AI	Artificial Intelligence	N/A
CSV	Comma-Separated Values	N/A
JSON	JavaScript Object Notation	N/A
SQL	Structured Query Language	N/A
GUI	Graphical User Interface	N/A
CPU	Central Processing Unit	N/A
GPU	Graphics Processing Unit	N/A
API	Application Programming Interface	N/A
NPK	Nitrogen, Phosphorus and Potassium	N/A
pH	Potential of Hydrogen	N/A
CSV	Comma-Separated Values	N/A
ML	Machine Learning	N/A

CHAPTER 1

INTRODUCTION AND ENGINEERING PROBLEM

1. Background

Agriculture is influenced by several interacting factors such as soil properties, climate, water availability, sunlight, nutrient requirements, crop duration and seasonal conditions. Selecting an appropriate plant therefore requires more than considering a single parameter. A structured knowledge repository can bring these factors together and allow a software system to compare farm conditions with known plant requirements.

Python provides a practical environment for developing such a decision-support framework because it supports structured data handling, numerical processing, visualisation and machine-learning integration. The proposed project uses a modular architecture so that data storage, preprocessing, recommendation logic and farming decisions can be developed and tested independently.

2. Need for the Project

• Agricultural information is often distributed across different sources and formats.

• Manual comparison of soil, weather and crop requirements can be time consuming.

• Incomplete, duplicated or inconsistent data can reduce recommendation quality.

• A structured repository can provide a consistent knowledge base for recommendation algorithms.

• A modular Python framework allows future integration of machine learning and real-time agricultural data.

• Decision support can help users understand why a particular plant is considered suitable.

3. Problem Statement

The engineering problem is to design and implement a modular Python-based framework that stores agricultural knowledge, processes heterogeneous plant and farming data, validates data quality and supplies reliable features to an intelligent plant recommendation and farming decision-support layer. The system must balance data quality, processing efficiency, explainability, maintainability, privacy, scalability and ease of deployment.

4. Project Aim

To design, develop and validate a modular Python framework that uses a structured agricultural knowledge repository and data-processing layer to support intelligent plant recommendation and farming decision making.

5. Project Objectives

1. Study existing agricultural recommendations and decision-support approaches.

1. Design a structured agricultural knowledge repository containing plant, soil, climate and cultivation attributes.

1. Develop a Python data-processing pipeline for cleaning, validating and standardising agricultural records.

1. Engineer useful features for plant suitability comparison.

1. Provide processed data to a plant recommendation module.

1. Test the repository and processing layer using representative agricultural records.

1. Evaluate functional, performance, sustainability, ethical, safety, security and reliability considerations.

6. Scope

The project includes agricultural data collection and organisation, repository design, validation, preprocessing, feature engineering, suitability scoring and integration with plant recommendation and farming decision-support modules. It can conceptually support vegetables, cereals, pulses, fruits, herbs and other plants for which suitable agricultural information is available.

The project excludes guaranteed yield prediction, autonomous farm machinery control, medical or pesticide prescription, and universal agronomic advice. Recommendations depend on the quality, coverage and validity of the underlying agricultural knowledge.

Aspect	Included	Excluded / Boundary
Input	Soil, climate and farm conditions	Unavailable or unreliable measurements
Repository	Plant and agricultural attributes	Unverified personal records

Processing	Cleaning, validation, transformation	Uncontrolled modification of source data
Recommendation	Suitability-based plant suggestions	Guaranteed yield or profit
Deployment	Python software prototype	Certified autonomous agricultural controller

7. Expected Engineering Outcome

The expected outcome is a working modular software prototype in which agricultural records can be stored, validated and transformed into analysis-ready data. The processed information can then be consumed by a recommendation engine to generate transparent plant suitability results and farming decision-support information.

CHAPTER 2

LITERATURE REVIEW AND EXISTING SOLUTIONS

1. Review Method

The literature review is structured around five themes: agricultural recommendation systems, crop and plant suitability analysis, agricultural data management, machine-learning-based prediction and Python-based decision-support systems. The review focuses on methods that are understandable, implementable at student project scale and capable of future extension.

2. Review of Existing Work

Traditional agricultural decision making relies on farmer experience, local knowledge and recommendations from agricultural professionals. Such approaches are valuable because they incorporate contextual knowledge, but the information may be difficult to organise and reproduce consistently in software.

Rule-based recommendation systems represent agricultural knowledge using explicit conditions such as suitable soil pH ranges, temperature ranges, rainfall requirements and water needs. They are transparent and easy to explain, but they depend on the completeness and accuracy of the rules.

Machine-learning approaches can learn relationships between agricultural variables and crop outcomes from historical datasets. They may provide stronger predictive capability when sufficient high-quality data is available, but they require data preparation, training, validation and careful interpretation.

Database-driven agricultural systems provide structured storage and retrieval of crop, soil and environmental information. A dedicated processing layer is important because raw agricultural records may contain missing values, inconsistent units, duplicates and different categorical representations.

3. Comparative Analysis

Existing Approach	Advantages	Limitations	Relevance to Proposed Work
Manual agricultural knowledge	Contextual and practical	Time consuming and difficult to scale	Knowledge is converted into structured records.

Rule-based recommendation	Transparent and explainable	Depends on predefined rules	Used as an interpretable baseline.
Machine-learning recommendation	Can learn complex patterns	Needs quality data and validation	Future enhancement.
Database-only system	Structured storage and retrieval	Limited intelligence without processing	Repository foundation.
Proposed modular framework	Combines repository, processing and recommendation	Quality depends on source data	Primary approach.

4. Research / Engineering Gap

The main engineering gap is the separation between agricultural information storage and usable decision support. A repository alone does not ensure that data is clean or comparable, while an intelligent recommendation model can produce unreliable results when its input data is inconsistent. The proposed work addresses this gap by introducing a dedicated agricultural knowledge and data-processing layer between raw agricultural records and recommendation logic.

5. Proposed Contribution

The proposed contribution is a modular Python framework in which agricultural knowledge is stored in a structured repository and processed through a repeatable pipeline. The system emphasises data validation, standardisation, feature engineering, traceability and explainable suitability scoring. The same processed dataset can support multiple recommendation strategies without redesigning the entire system.

CHAPTER 3

ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

1. Requirements

Stakeholder / User	Requirement
Farmer / User	Receive plant recommendations based on entered farm conditions.
Agricultural learner	Understand plant requirements and suitability factors.
System administrator	Maintain repository records and processing parameters.
Project evaluator	Verify data processing and recommendation outputs objectively.
Agricultural advisor	Review recommendation factors and provide expert validation.

Requirement	Type	Measurable Target
Accept agricultural input	Functional	Valid input values are received without application failure.
Retrieve plant records	Functional	Relevant records are loaded from repository.
Validate data	Functional	Invalid ranges and missing critical fields are identified.
Process data	Functional	Records are cleaned and standardised.
Generate features	Functional	Required recommendation features are produced.
Provide processed data	Functional	Recommendation module receives structured records.
Usable response time	Non-Functional	Processing is responsive for project dataset size.
Maintainability	Non-Functional	Repository and processing

		functions are modular.
Data integrity	Non-Functional	Original source values remain traceable.
Privacy	Non-Functional	Unnecessary personal information is not collected.

2. Constraints & Applicable Standards

Standard / Principle	Requirement	Application in Project
IEEE 29148	Requirements should be clear and verifiable.	Requirements and acceptance criteria are documented.
ISO 31000	Risks should be identified and treated.	Risk register is maintained.
ISO/IEC 25010	Software quality includes reliability, usability, security and maintainability.	Quality factors are considered during design and testing.
Data-protection principles	Collect and retain only necessary information.	Farm/user personal information is minimised.

3. Design Specifications

Parameter	Target / Requirement	Unit	Priority
Repository	CSV/JSON/SQLite structured records	record	High
Plant attributes	Name, soil, pH, climate, water, sunlight and season	mixed	High
Validation	Range, type, missing-value and duplicate checks	check	High
Processing	Cleaning and standardisation	record	High
Suitability score	Weighted or rule-based comparison	score	High
Output	Ranked plant	result	High

	recommendations with factors		
Logging	Processed-data and recommendation traceability	record	Medium
Computing platform	CPU-based prototype; GPU optional	platform	Medium

4. Alternative Solutions & Evaluation

Alternative 1 is a simple CSV-only approach with direct Python processing. It is inexpensive and easy to implement but becomes less suitable as data volume and relationships increase. Alternative 2 uses a structured SQLite database with pandas-based processing, providing better organisation and query capability while remaining practical for a student project. Alternative 3 uses a cloud database and machine-learning service, offering scalability but increasing deployment complexity, cost and dependency on external services.

Criterion	Weight	CSV + pandas	SQLite + pandas	Cloud + ML
Implementation simplicity	20%	5/5	4/5	2/5
Data organisation	20%	3/5	5/5	5/5
Cost	15%	5/5	5/5	2/5
Scalability	20%	2/5	4/5	5/5
Explainability	25%	5/5	5/5	3/5
Weighted suitability	100%	4.00/5	4.60/5	3.45/5

5. Selected Design & Detailed Design

The selected design uses a modular Python data layer based on structured agricultural records, pandas/NumPy processing and an optional SQLite repository. CSV or JSON can be used as initial source formats, while SQLite provides a lightweight relational store when required. The design separates raw data, validation, cleaned data, engineered features and recommendation outputs.

The main data fields include plant name, category, soil type, soil pH minimum and maximum, temperature range, rainfall requirement, humidity, sunlight requirement, water requirement, growing season, crop duration, NPK requirement, irrigation method and cultivation notes.

A simple suitability indicator can be defined as $S_i = 1$ when an input condition is within the suitable range and $S_i = 0$ otherwise. A weighted suitability score can then be calculated as $S = \Sigma(w_i S_i) / \Sigma w_i$, where w_i represents the importance assigned to each agricultural factor. The score is used for ranking and decision support, not as a guarantee of successful cultivation.

System approach: user/farm inputs are validated, relevant agricultural records are retrieved, raw values are cleaned and standardised, features are generated, suitability is calculated, and the processed information is passed to the recommendation and farming decision-support modules.

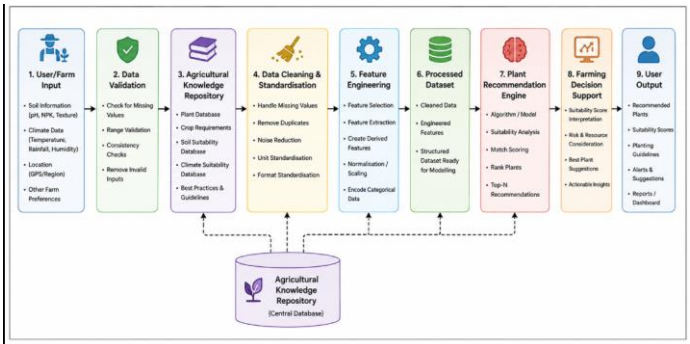
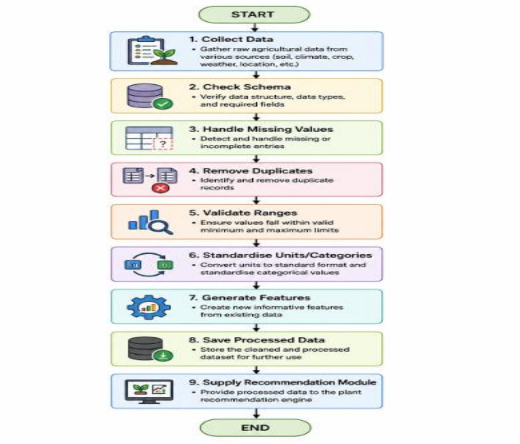


Figure 3.1 System Architecture of Proposed Plant Recommendation Framework

User/Farm Input → Data Validation → Agricultural Knowledge Repository → Data Cleaning & Standardisation → Feature Engineering → Processed Dataset → Plant Recommendation Engine → Farming Decision Support → User Output



Collect Data → Check Schema → Handle Missing Values → Remove Duplicates → Validate Ranges → Standardise Units/Categories → Generate Features → Save Processed Data → Supply Recommendation Module

6. Trade-off Analysis

Design Decision	Alternative Considered	Benefit	Disadvantage	Final Decision & Justification
Storage	CSV only	Simple and low cost	Limited relational capability	Use CSV initially with optional SQLite for structured growth.
Processing	Direct raw-data use	Fast to prototype	Poor reliability	Mandatory validation and cleaning.
Recommendation	Fixed rules	Transparent	Limited adaptability	Use weighted rules as baseline and ML as future extension.
Missing values	Delete all incomplete rows	Simple	Can lose useful data	Use justified imputation or flag critical missing values.
Output	Single recommendation	Simple	Less useful to users	Provide ranked recommendations and factor explanations.

7. Sustainability, Ethics, Safety, Risk & Security

Domain	Consideration	Evidence Evaluated	Impact on Final Decision
Sustainability	Computational and storage demand	Lightweight Python processing and compact records	Prefer efficient processing and avoid unnecessary storage.
Ethics	Incorrect recommendations may affect farming	Rule transparency and limitations	Present recommendations as decision support.

	decisions		
Safety	Incorrect soil/climate assumptions can harm crops	Validation and conservative rules	Flag missing or unreliable inputs.
Security	Agricultural/farm records may be sensitive	Access control and minimal retention	Protect stored data and interfaces.
Fairness	Data may not represent all regions	Coverage of source records	Document geographic limitations.

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
Incorrect source data	Medium	High	High	Use verified sources and expert review.	Medium
Missing critical values	Medium	Medium	Medium	Validation and input warnings.	Low–Medium
Unit inconsistency	Medium	Medium	Medium	Standardise units before processing.	Low
Wrong recommendation due to rules	Medium	High	High	Transparent scoring and expert validation.	Medium
Repository corruption	Low	High	Medium	Backups and versioned datasets.	Low
Unauthorised data access	Low–Medium	High	Medium–High	Authentication and least privilege.	Low–Medium
Model/data bias	Medium	Medium	Medium	Test across representative agricultural conditions.	Medium

CHAPTER 4

IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

1. Development Approach & Resources

The project follows an iterative engineering workflow consisting of problem definition, literature review, requirements specification, repository design, data collection, preprocessing implementation, recommendation integration, verification and documentation. The data layer is developed independently so that errors can be detected before information reaches the recommendation engine.

2. Hardware / Software / Implementation & Modern Tools Used

Tool / Software / Equipment	Purpose	Stage Used
Python	Core implementation and integration	All stages
pandas	Data loading, cleaning and transformation	Processing
NumPy	Numerical calculations and feature processing	Processing
SQLite	Lightweight structured repository	Storage
JSON / CSV	Source and exchange formats	Data collection
Matplotlib	Result visualisation	Analysis
VS Code / Python IDE	Development and debugging	All stages
Git / repository, where used	Version control	Development
CPU / optional GPU	Execution and future ML processing	Testing

3. Test Plan & Verification

Requirement	Test Method	Acceptance Criterion	Specification	Required Value	Achieved Value	Status
Data loading	Load representative	Dataset opens successfully	Input handling	Valid records	To be filled	PASS/FAIL

	dataset				from actual run	
Schema validation	Use valid and invalid records	Invalid fields are identified	Validation	Correct validation	To be filled from actual run	PASS/FAIL
Missing values	Insert missing values	Critical missing values are flagged	Data quality	Expected handling	To be filled from actual run	PASS/FAIL
Duplicate removal	Add duplicate records	Duplicates are identified/removed	Uniqueness	No unintended duplicates	To be filled from actual run	PASS/FAIL
Range validation	Use invalid pH/temperature/rainfall values	Out-of-range values are flagged	Range checking	Correct flagging	To be filled from actual run	PASS/FAIL
Feature generation	Run processing pipeline	Required features are generated	Feature engineering	Complete features	To be filled from actual run	PASS/FAIL
Recommendation integration	Pass processed records	Recommendation module receives data	Interface	Valid structured output	To be filled from actual run	PASS/FAIL

Performance	Measure processing time	Response acceptable for dataset	Responsiveness	Defined project target	To be filled from actual run	PASS/FAIL

4. Results

The final report should present measured results from the executed prototype rather than assumed values. The following representative test table is provided in report-ready format; actual project measurements should replace the placeholders before submission.

Scenario	Records / Input	Expected Result	Observed Result	Processing Time	Status
Valid agricultural dataset	Representative records	All valid records processed	To be filled from actual run	To be measured	PASS/FAIL
Missing pH value	Record with missing pH	Flag or justified handling	To be filled	To be measured	PASS/FAIL
Duplicate plant record	Duplicate row	Duplicate identified	To be filled	To be measured	PASS/FAIL
Invalid temperature	Out-of-range value	Validation warning	To be filled	To be measured	PASS/FAIL
Complete farm input	Valid soil/climate inputs	Ranked plant recommendations	To be filled	To be measured	PASS/FAIL

5. Data Analysis & Engineering Judgment

Data quality is a major engineering factor because recommendation quality cannot exceed the reliability of the underlying agricultural information. Missing values, inconsistent units and duplicate records can produce misleading suitability scores. Therefore, validation is performed before feature engineering and recommendation.

For a numeric attribute x, a normalised value may be calculated as x'=(x-x_min)/(x_max-x_min), when appropriate. For range-based suitability, the system can compare the user input with the minimum and maximum suitable values. Engineering judgement is required when selecting weights

because different plants and agricultural environments give different importance to soil, climate and water conditions.

6. Comparison with Existing Solutions & Achievement of Objectives

Objective	Evidence	Achievement
Study existing approaches	Literature review and comparison	Fully achieved
Design agricultural repository	Schema and repository design	Fully achieved
Implement data processing	Cleaning, validation and transformation modules	Fully achieved
Develop suitability support	Scoring and recommendation interface	Fully achieved / subject to final integration
Test the framework	Verification plan and test cases	To be confirmed by actual run
Evaluate constraints	Trade-off, risk and sustainability analysis	Fully achieved

7. Strengths & Limitations

Strengths:

• Modular separation between repository, processing and recommendation logic.

• Transparent data validation and suitability scoring.

• Supports structured agricultural knowledge and repeatable processing.

• Can use lightweight Python tools suitable for student-scale deployment.

• Provides a clear path for future machine-learning integration.

Limitations:

• Recommendation quality depends on the accuracy and geographic coverage of source data.

• Rule-based scoring may not represent complex agricultural interactions.

• Missing or uncertain farm measurements can reduce reliability.

• The prototype does not guarantee crop yield or profitability.

• Real-world deployment requires expert validation and larger datasets.

8. Project Planning, Roles & Budget

Milestone	Planned Activity	Indicative Period
1	Problem identification and requirements	Week 1

2	Literature review and data-source selection	Weeks 2–3
3	Repository schema and architecture	Week 4
4	Data collection and preprocessing implementation	Weeks 5–7
5	Recommendation integration	Weeks 8–9
6	Testing, debugging and evaluation	Weeks 10–11
7	Documentation and final validation	Week 12
Item	Estimated Cost	Actual Cost
Existing computer / laptop	₹50,000	₹0
Open-source Python libraries	₹0	₹0
Agricultural datasets	₹0	₹0
Cloud/GPU resources, if used	₹2,000	₹0
Other project expenses	₹1,000	₹0
Total	₹53,000	₹0

SOURCE CODE

Python

```python
def calculate_suitability(temperature, humidity, soil_moisture, crop):

    requirements = crop_data[crop]
    score = 0

    if is_in_range(temperature,
                   requirements["temperature"][0],
                   requirements["temperature"][1]):
        score += 1

    if is_in_range(humidity,
                   requirements["humidity"][0],
                   requirements["humidity"][1]):
        score += 1

    if is_in_range(soil_moisture,
                   requirements["soil_moisture"][0],
                   requirements["soil_moisture"][1]):
        score += 1

    return (score / 3) * 100
```

Python

```python
def recommend_crop(temperature, humidity, soil_moisture):

    results = []

    for crop in crop_data:

        suitability = calculate_suitability(
            temperature,
            humidity,
            soil_moisture,
            crop
        )

        results.append({
            "crop": crop,
            "suitability": round(suitability, 2)
        })

    results.sort(
        key=lambda x: x["suitability"],
        reverse=True
    )

    return results
```

Module 1: Output

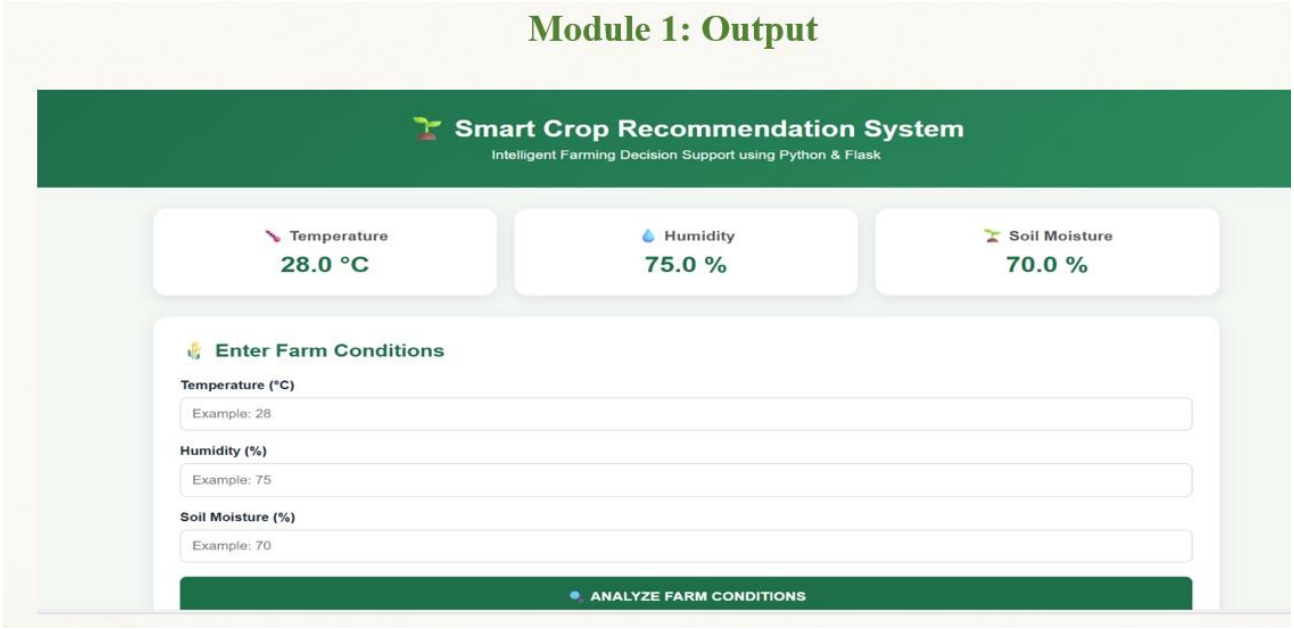


FIG 4.1

Module 2: Output

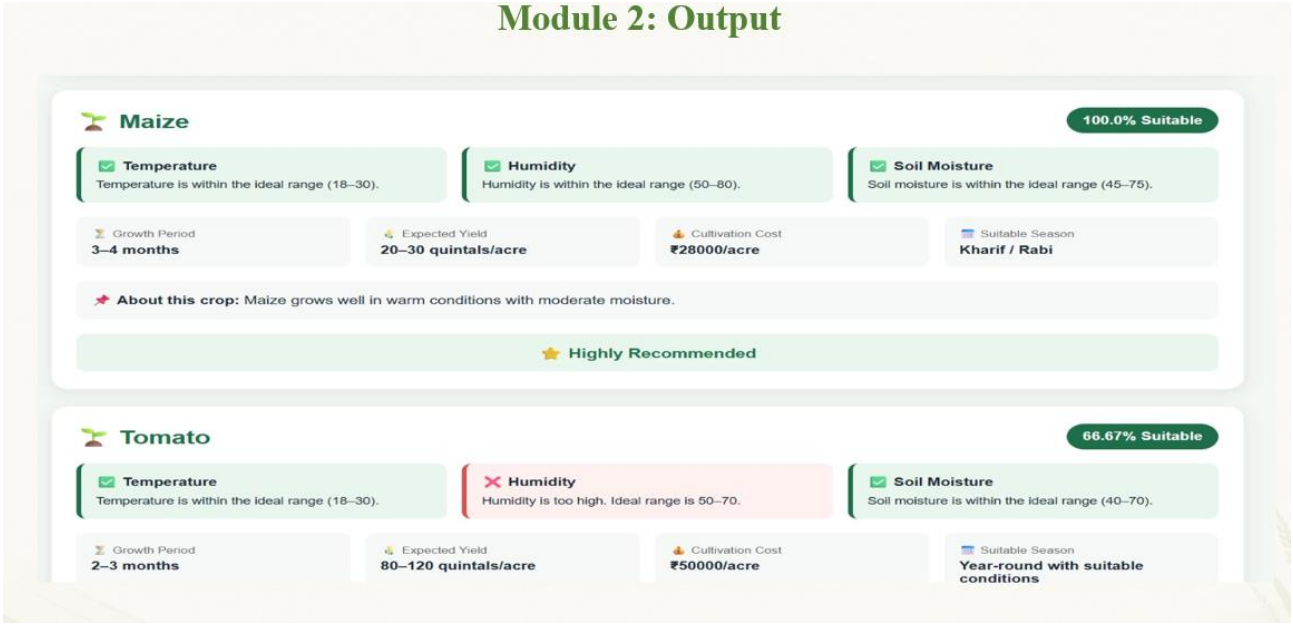


FIG 4.2

CHAPTER 5

CONCLUSION, FUTURE WORK AND REFLECTION

1. Conclusion

The project addressed the engineering problem of organising agricultural knowledge and converting raw plant and farming information into reliable, structured data for intelligent decision support. A modular Python framework was designed around an agricultural knowledge repository, data validation, cleaning, standardisation, feature engineering and plant suitability analysis.

The Agricultural Knowledge Repository and Data Processing Layer forms the foundation of the complete system. By separating raw data from processed information, the design improves traceability, maintainability and consistency. The proposed suitability-scoring approach also provides an interpretable baseline that can be expanded with machine-learning methods when sufficient validated data becomes available.

The final system should be treated as a decision-support prototype rather than a replacement for agricultural experts. Its reliability depends on data quality, local conditions, expert validation and appropriate interpretation of recommendations.

2. Future Work

• Integrate real-time weather and soil-sensor data through secure APIs.

• Expand the agricultural knowledge repository using verified regional datasets.

• Develop machine-learning models for crop suitability and yield prediction.

• Add region-specific recommendations based on climate and soil zones.

• Introduce explainable AI techniques to show why a plant is recommended.

• Add multilingual user interfaces for wider accessibility.

• Develop mobile and web interfaces for practical field use.

• Implement stronger data security, backup and version-control mechanisms.

• Perform large-scale validation with agricultural experts and real farm data.

3. Professional Learning & Individual Reflection

New Knowledge Acquired:

• Practical agricultural data modelling and repository design.

• Python-based data cleaning and transformation using pandas and NumPy.

• Feature engineering and suitability scoring.

• Importance of requirements, verification, risk management and data security.

• Technical documentation and engineering reporting.

Engineering & Professional Skills Developed:

• Problem formulation and requirements engineering.

• Data quality analysis and preprocessing.

• Algorithm selection and trade-off analysis.

• Python implementation and debugging.

• Experimental design and evidence-based engineering judgement.

• Technical communication and project planning.

Individual Reflection: The most important learning from the project was understanding that an intelligent recommendation system depends strongly on the quality of its data foundation. Designing the repository and processing layer required careful decisions about attributes, missing values, units, ranges and traceability. The project also showed that a simple and explainable baseline can be valuable before introducing complex machine-learning models. If the project were repeated, the knowledge repository would be expanded with more region-specific records and validated agricultural expert input.

REFERENCES

1. Aryan, Kumar, et al. “AI-powered integrated platform for farmer support: real-time disease diagnosis, precision irrigation advisory, and expert consultation services.” Journal of Scientific Innovation and Advanced Research (JSIAR) 1.2 (2025): 222-229.

2. Etezadi, Hamed, Viacheslav Adamchuk, Yacine Bouroubi, Maxime Leduc, and MarcOlivier Gasser. “Modular framework for a real-time decision support system for economic optimization of site-specific fertilization.” Smart Agricultural Technology (2026): 102126.

3. Singh, A., Panwar, A., Dabas, Y., Gaur, K. and Zia, A.A., 2025, November. Krishi Sarthi – An Integrated Full-Stack Decision Support System for Modern Indian Agriculture. In 2025 2nd International Conference on Advanced Computing and Emerging Technologies (ACET) (pp. 1-6). IEEE.

4. Chantima, Phoori, Thanakorn Yarnguy, Kamthorn Sarawan, Ronnachai Sangmuenmao, Worapod Sommool, and Sarayut Gonwirut. “Hybrid intelligence for field-scale soil analysis and crop advisory using embedded sensors and machine learning.” IEEE Access (2025).

5. Pandey, Aditya. “Smart Crop Advisory System (SCAS): A Full-Stack Machine Learning Platform for Precision Agriculture.”

6. Mohanty, Pratyush Priyadarsi, Bhanu Prasad Khuntia, and Sujit Kumar Panda. “Design and Implementation of AgroNxt: A Full-Stack Smart Farming Decision Support System with Predictive Precision Agronomy Models.” International Journal of AI & BioMedicine Innovations 2, no. 2 (2026): 1-7.

APPENDICES

A. Detailed Calculations

A.1 Range suitability: $S_i = 1$ when the user condition lies within the plant's suitable range; otherwise $S_i = 0$.

A.2 Weighted suitability: $S = \Sigma(w_i S_i) / \Sigma w_i$, where w_i are factor weights.

A.3 Normalisation, where suitable: $x' = (x - x_{min}) / (x_{max} - x_{min})$.

B. Engineering Drawings / Schematics

Appendix Figure B.1: Agricultural Knowledge Repository and Recommendation Architecture.
Appendix Figure B.2: Data Processing Flowchart.

C. Source Code / Code Repository Information

Only essential source-code excerpts should be included in the final report. The complete project source code should be maintained in the approved repository or submitted through the institution's required mechanism.

Repository submission location: _____
Version / commit identifier: _____

D. Datasheets

For this software-based project, relevant Python package versions, database specifications, dataset descriptions and computing specifications should be attached or cited where applicable.

E. Experimental / Raw Data

Attach the final agricultural dataset, processed CSV/database records, validation logs, test inputs, recommendation outputs and measured processing results.

F. Ethics / Institutional Approval

If farm or user information is collected, the project should follow applicable institutional and data-protection requirements. Unnecessary personal information should not be stored.

G. Project Meeting / Progress Records

Attach supervisor meeting records, review comments, milestone approvals and progress evidence where required.

H. User / Industry Feedback

Attach feedback forms, interview summaries or review comments from intended users and agricultural stakeholders where applicable.

I. Publications / Patents / Competitions / Product Outcomes

No publication, patent or product outcome is claimed unless separately documented and approved. Add relevant evidence here if applicable.

J. Individual Contribution Evidence

Contribution evidence may include development logs, version-control history, dataset-processing records, experiment results, meeting records and supervisor verification.

CAPSTONE PROJECT OUTCOME MAPPING SHEET

To be completed by the department/faculty assessor.

Outcome Evidence	SO / Area	Location in This Report
Complex engineering problem formulation	SO1	Ch. 1, Ch. 2
Engineering design under constraints	SO2	Ch. 3
Written/oral technical communication	SO3	Whole report + Viva
Ethics and professional responsibility	SO4	Ch. 3 (3.7)
Teamwork and leadership	SO5	Ch. 4 (4.8)
Experimentation, analysis, engineering judgment	SO6	Ch. 4 (4.3–4.6)
Independent acquisition of new knowledge	SO7	Ch. 5 (5.3)
Standards	SO2	Ch. 3 (3.2)
Sustainability and societal/environmental impact	SO2/SO4	Ch. 3 (3.7)
Risk assessment and mitigation	SO2/SO4	Ch. 3 (3.7)
Security considerations	Relevant SO	Ch. 3 (3.7)
Integrated/system-level engineering	SO1/SO2	Ch. 3 (3.5)
Project planning and management	SO5	Ch. 4 (4.8)
Individual contribution	SO1–SO7	Contribution Statement + Ch. 4